Εισαγωγή και Αντιμετώπιση των Hardware Trojan Κυκλωμάτων σε FPGA

Τμήμα Μηχανικών Η/Υ και Πληροφορικής, Πανεπιστήμιο Πατρών, Πάτρα
Ασφάλεια σε Υλικό
AM:1084672
up1084672@ac.upatras.gr

Abstract
Η παρούσα εργασία παρουσιάζει μία νέα μέθοδο επίθεσης μέσω Hardware Trojans σε FPGA συστήματα, εστιάζοντας στην αθόρυβη εισαγωγή και ενεργοποίηση τους μέσω παραποιημένων EDA εργαλείων. Σε αντίθεση με κλασικές τεχνικές, ο Trojan που προτείνεται εισάγεται μετά τη φάση σύνθεσης και παραμένει αδρανής έως τη φάση προγραμματισμού του FPGA, παρακάμπτοντας κάθε στάδιο επαλήθευσης. Η επίθεση βασίζεται σε τεχνικές αποσύνδεσης και επανασύνδεσης (TPIP), επιτρέποντας τον Trojan να ενεργοποιηθεί μόνο όταν πληρούνται συγκεκριμένες συνθήκες, όπως η χρήση τροποποιημένων εργαλείων προγραμματισμού. Μέσω open-source εργαλείων και πειραμάτων με κυκλώματα AES, αποδεικνύεται ότι το Trojan μπορεί να διαρρεύσει κρυφά κλειδιά χωρίς να αλλοιώνει τη λειτουργικότητα του κυκλώματος. Η εργασία υπογραμμίζει την αδυναμία των τρεχουσών τεχνικών ανίχνευσης, ενώ ταυτόχρονα προτείνει μελλοντικές κατευθύνσεις για ανίχνευση με χρήση τεχνικών ανάγνωσης (read-back) και machine learning. Η μικρή κατανάλωση και το αμελητέο μέγεθος του Trojan καθιστούν την επίθεση εξαιρετικά δύσκολη στον εντοπισμό, προσφέροντας ένα επικίνδυνα αποδοτικό μέσο παραβίασης της ασφάλειας υλικού.

Keywords: Hardware Trojan, TPIP Activation, Reverse Engineering, EDA Tools, Post-configuration Attack, Low-power Trojan, Proof-Carrying Hardware (PCH), Machine Learning Detection, AES Cryptographic Core

1 Σκοπός και Στόχος

1.1 Σκοπός

Ο σκοπός της εργασίας είναι η παρουσίαση ενός Hardware Trojans που έχει αρκετές διαφορές από τα συνηθισμένα software trojans που γνωρίζουμε. Έχει τη δυνατότητα να εφαρμοστεί σε ένα ηλεκτρονικό κύκλωμα που στη δική μας περίπτωση θα είναι σε FPGA όπου μπορεί να αλλάξει όλη τη ροή του κυκλώματος και να μην ανιχνευτεί. Η εργασία αρχικά σκοπεύει να μας εξηγήσει τα τρόποι συμπεριφοράς του Trojans αλλά και τα στάδια, όπου συμπεριφέρεται σε διαφορετικές λειτουργίες (αδράνεια, ενεργό). Περιγράφεται αναλυτικά πως το Trojan καταφέρνει να μην ανιχνευτεί από το σχεδιαστή αλλά και τη διαδικασία που χρειάζεται να κάνει έτσι ώστε να μπορεί να αλλάξει το bitstream του FPGA.

Στη συνέχεια έχουμε και τους επιτιθέμενους (attackers) που η εργασία σκοπεύει να μας περιγράφει την αντίστροφη μηχανική (reverse engineering) που εφαρμόζεται και ο τρόπος επίθεσης που γίνεται όπου το μεγαλύτερο μέρος της επίθεσης γίνεται σε EDA (Electronic Desing Automation) Tools που ο attacker τροποποιεί αυτά τα εργαλεία.

Για να είμαστε ποιο συγκεκριμένη έχουμε και προηγούμενες εργασίες που με κάποιο τρόπο μας απαντάνε την ερώτηση για ποιο λόγο έγινε η εργασία και τι σκοπεύει. Η πρώτη εργασία που έγινε το 2013 τους Chakraborty RS, Saha I και Palchaudhuri A, Naik GK που παρουσίασαν τρόπους για την εισαγωγή Trojans απευθείας σε μη κρυπτογραφημένα bitstreams μέσω εξειδικευμένων εργαλείων. Εκεί, διακρίθηκαν δύο είδη Trojans:

- Τύπου I, τα οποία δεν συνδέονται με το αρχικό κύκλωμα (μη λειτουργικά) και απλώς τοποθετούνται σε ελεύθερους πόρους της FPGA.
- Τύπου II, τα οποία συνδέονται με το αρχικό κύκλωμα και απαιτούν σημαντική γνώση του σχεδιασμού για την υλοποίησή τους.

Ωστόσο, τέτοιες επιθέσεις έχουν σοβαρούς περιορισμούς, ειδικά σε περιπτώσεις όπου το bitstream είναι κρυπτογραφημένο ή έχουν τοποθετηθεί πλασματικά κυκλώματα (dummy logic) για να μπλοκάρουν την εισαγωγή επιπλέον λογικής.

Αντίστοιχα, το 2016 οι Krieg C, Wolf C και Jantsch A πρότειναν μια πιο εξελιγμένη επίθεση, όπου το Trojan (με τη μορφή "malicious LUT") εισάγεται στο σχεδιασμό, παραμένει ανενεργό κατά τη φάση ανάπτυξης, και ενεργοποιείται μόνο όταν το bitstream γράφεται. Αν και αυτή η μέθοδος είναι πιο αθόρυβη, μπορεί πλέον να εντοπιστεί με πιο εξελιγμένες τεχνικές επαλήθευσης, όπως η Proof-Carrying Hardware (PCH).

Η παρούσα εργασία προχωράει ένα βήμα παραπέρα, παρουσιάζοντας έναν νέο τύπο Trojan που όχι μόνο παραμένει ανενεργός καθ' όλη τη σχεδίαση αλλά και εντός του ίδιου του bitstream, χωρίς να προκαλεί ύποπτες μεταβολές. Ο Trojan ενεργοποιείται μόνο μετά τον προγραμματισμό της FPGA, μέσω ενός πειραγμένου εργαλείου προγραμματισμού, καθιστώντας τον απολύτως αόρατο για όλες τις γνωστές μεθόδους ανίχνευσης.

Ουσιαστικά, η έρευνα σκοπεύει να μας δείξει πως αποτυγχάνουν τα τρέχοντα αντίμετρα λόγω της αδράνειας και τις αποσυνδέσεις που κάνει το Trojan στα στάδια όπως προ-διαμόρφωση (pre-configuration), bitstream-level verification και λειτουργική προσομοίωση. Όπου οι ερευνητές τις εργασίας σκοπεύουν να έχουν μία νέα έρευνα για τους τρόπους

1.2 Στόχος

Παρουσιάσει μια νέα, εξαιρετικά κρυφή μέθοδο εισαγωγής και ενεργοποίησης Hardware Trojans (κακόβουλων κυκλωμάτων) σε FPGA (Field Programmable Gate Arrays). Το βασικό της επιχείρημα είναι ότι ακόμα και τα πιο προηγμένα εργαλεία επαλήθευσης (verification) δεν μπορούν να ανιχνεύσουν αυτό το Trojan, επειδή παραμένει ανενεργό μέχρι να προγραμματιστεί η συσκευή.

Με τα βήματα που παρουσιάζονται στο 5ο κομμάτι της εργασίας, μπορούμε να καταλάβουμε εύκολα ότι η εργασία είναι μία έρευνα που προσπαθεί να μας εξηγήσει βήμα βήμα όλη τη διαδικασία παρέχοντας και τα δεδομένα αλλά και κάθε υλικό που χρειάστηκε στη διαδικασία του πειράματος με στόχο να δείξει ξεκάθαρα τη μελέτη και να πείσει κόσμο για τις επόμενες γενιές.

Βέβαια, στο τέλος της εργασίας οι ερευνητές παρουσιάζουν και τον επόμενο τους στόχο που είναι ουσιαστικά είναι η διαδικασία της ανίχνευσης αυτών των επιθέσεων. Με καλύτερα λόγια, να μελετήσουν πιθανά αντίμετρα για την επίθεση που παρουσιάζουν, εστιάζοντας ιδιαίτερα σε τεχνικές επαλήθευσης μετά τον προγραμματισμό της FPGA (post-configuration verification). Μία από τις προτάσεις τους είναι η αξιοποίηση της λειτουργίας read-back, δηλαδή η δυνατότητα να «διαβαστεί» το περιεχόμενο του προγραμματισμένου κυκλώματος από την ίδια τη συσκευή. Μέσω αυτής της λειτουργίας, θα μπορούσε να συγκριθεί το πραγματικό κύκλωμα που έχει προγραμματιστεί με το αναμενόμενο, εντοπίζοντας έτσι πιθανές κακόβουλες τροποποιήσεις όπως το Trojan που περιγράφουν.

2 Μεθοδολογία

Σε αυτό το κομμάτι της εργασίας θα παρουσιάσουμε όλη τη μεθοδολογία αλλά και τις συνοπτικές προσεγγίσεις της προηγούμενης ενότητας με αποτέλεσμα να έχουμε μία σωστή και έγκυρη παρουσίαση όλης της εργασίας.

Αρχικά θα περιγράψουμε την ενεργοποίηση και το μοντέλο απειλής, όπου το Trojan τοποθετείται με τέτοιο τρόπο ώστε να παραμένει ανενεργό (κρυφό) ακόμα και μέσα στο τελικό bitstream που προγραμματίζει τη FPGA. Έτσι, δεν μπορεί να εντοπιστεί από τις υπάρχουσες τεχνικές επαλήθευσης σε επίπεδο bitstream.

Στη συνέχεια, θα μεταβούμε στην επίθεση, εξηγούμε αναλυτικά πως γίνεται η επίθεση, υποστηρίζοντας την επίθεση με τους αλγόριθμους που χρησιμοποιήθηκαν στα 2 στάδια και με τους

μεθόδους όπου περιγράφεται η συμπεριφορά του κυκλώματος και η αλλαγή των εισόδων ανάλογα με την κατάσταση της επίθεσης.

Τέλος, εξηγούμε τα εργαλεία σχεδίασης για την κρυπτογράφησης αλλά και τις μεθοδολογίες επίθεσης στο πρακτικό μέρος της έρευνας. Όπου χρησιμοποιείται open-source εργαλεία που με αυτόν τον τρόπο οι σχεδιάσεις είναι ποιο σαφές και κατανοητές. Εκτός από τα open-source περιέχουμε και ένα κύκλωμα που περιγράφεται η κρυπτογράφηση.

2.1 Ενεργοποίηση και Απειλή του Trojan

Το Trojan να αποσυνδέεται από το κύριο κύκλωμα όταν δημιουργείται το bitstream, οπότε φαίνεται εντελώς ακίνδυνο. Αργότερα, κατά τη διάρκεια ή μετά τον προγραμματισμό του FPGA, το εργαλείο προγραμματισμού επανασυνδέει κρυφά τον Trojan, ενεργοποιώντας το χωρίς να γίνει αντιληπτός. Επειδή το Trojan είναι ανενεργό τη στιγμή της δημιουργίας bitstream, ακόμη και οι προηγμένες μέθοδοι επαλήθευσης όπως το Proof-Carrying Hardware (PCH) θα πιστοποιούσαν εσφαλμένα τη σχεδίαση ως ασφαλή. Η διαδικασία όπου καταχωρούνται τα επικίνδυνα δεδομένα είναι στο αρχείο .blif όπου είναι μια μορφή αρχείου που χρησιμοποιείται για την αναπαράσταση ψηφιακών λογικών κυκλωμάτων. Έπειτα στο αρχείο ascii. κρύβεται το Trojan διακόπτοντας τη σύνδεσή του πριν προγραμματιστεί το FPGA. Στο συγκεκριμένο αρχείο είναι σε ασφαλές λειτουργία δηλαδή έχει διακοπεί η σύνδεση του από το επικίνδυνο κύκλωμα.

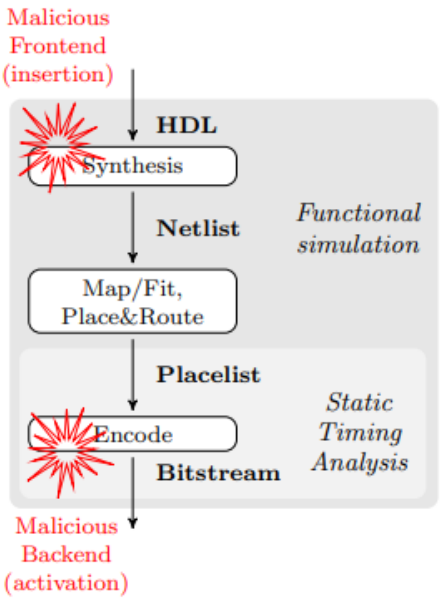
Στο μοντέλο που παρουσιάζουμε εμπιστευόμαστε στην εταιρία σχεδίασης (design house) και στους third-party IP προμηθευτές, δηλαδή αυτοί δεν είναι τα πρόσωπα που έχουν προσθέσει το Trojan μέσα στο κύκλωμα ώστε να μπορούν να κάνουν το χακάρισμα. Στη δική μας περίπτωση, οι developer του κυκλώματος δηλαδή οι σχεδιαστές του κυκλώματος σε EDA tools είναι οι ύποπτοι. Παρόλο που αναφέρεται και αναλυτικά και στο [1] για τις διαδικασίες που μπορεί να εφαρμοστεί ένας Trojan σε διαφορά στάδια, εμείς ενδιαφερόμαστε για τα EDA tools.

Για την απειλή που θα δημιουργήσουμε αρχικά θα πρέπει να ακολουθήσουμε μία παλιότερη έρευνα που βασίζεται η εργασία μας [2]. Όπως μπορούμε να παρατηρήσουμε και από την εικόνα 1, γίνεται η εισαγωγή και ενεργοποίηση ενός Trojan μέσα στην αλυσίδα εργαλείων σχεδίασης FPGA, χρησιμοποιώντας υπονομευμένα εργαλεία σχεδίασης. Εμείς βασιζόμενη σε αυτήν την εργασία, θα τοποθετήσουμε την εργασία αλλά και θα βελτιώσουμε την ιδέα.

Το Trojan ενεργοποιείται μόνο αν και το εργαλείο προγραμματισμού του FPGA του καταναλωτή και το εργαλείο τοποθέτησης/δρομολόγησης του σχεδιαστικού οίκου είναι υπονομευμένα (δηλαδή τροποποιημένα από τον επιτιθέμενο για να εκτελούν κακόβουλες ενέργειες).

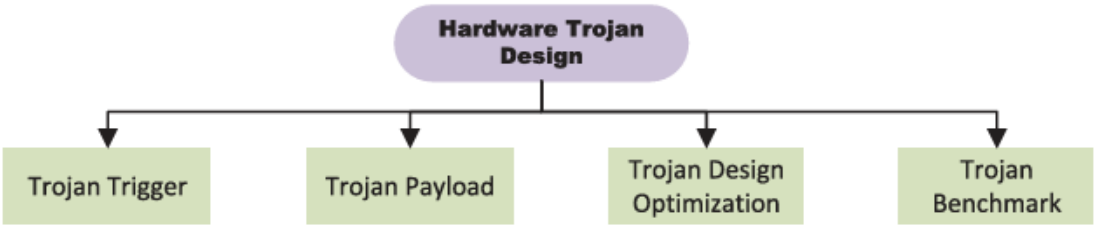
Αν μόνο ένα από τα εργαλεία είναι υπονομευμένο, τότε το Trojan παραμένει αδρανές, δηλαδή δεν ενεργοποιείται και η συσκευή FPGA λειτουργεί κανονικά χωρίς να δείχνει ότι έχει προσβληθεί από Trojan.

Το bitstream, πρέπει να είναι μη κρυπτογραφημένο ή το εργαλείο προγραμματισμού πρέπει να μπορεί να το αποκρυπτογραφήσει. Αν το bitstream είναι κρυπτογραφημένο και το εργαλείο δεν μπορεί να το αποκρυπτογραφήσει, η επίθεση δεν μπορεί να πραγματοποιηθεί. Η επίθεση στοχεύει μόνο τα εργαλεία προγραμματισμού των στόχων του, κάνοντάς το Trojan να παραμένει αδρανές και σχεδόν αόρατο στους πελάτες. Έτσι, το Trojan μπορεί να περάσει απαρατήρητο στις περισσότερες περιπτώσεις και να ενεργοποιηθεί μόνο όταν πληρούνται οι κατάλληλες συνθήκες.



Εικόνα 1. Αρχικό μονοπάτι που ακολουθεί η εργασία μας βασιζόμενα σε προηγούμενη εργασία.

Στη συνέχεια, παρατηρούμε ότι έχουμε τα άρθρα [3-5] που μιλάνε για τους μηχανισμούς ενεργοποίησης αλλά και τα χαρακτηριστικά του payload. Όπου η αρχική σχεδίαση του trojan είναι όπως μπορούμε να παρατηρήσουμε και στη εικόνα 2. Με περαιτέρω και νέες έρευνες η διαδικασία της σχεδίασης έγινε ποιο περίπλοκη όπως αναφέρεται και στη δική μας έρευνα, διαφορετικά από τις προηγούμενες έρευνες έχουμε μία νέα μέθοδο η οποία λέγεται connection-based conditional trigger. Σε αυτή τη νέα μέθοδο, καθορίζει αν ο ενεργοποιητής είναι πραγματικά συνδεδεμένος με το αρχικό κύκλωμα ή αποσυνδέεται προσωρινά για να παρακάμψει οποιουσδήποτε μηχανισμούς δοκιμών και ενεργοποιείται μετά την επανασύνδεση.



Εικόνα 2. Πρώτα βήματα σε σχεδίαση ενός Trojan σε υλικό

Αυτή η νέα μέθοδο ουσιαστικά βοηθά και σε παλιές αναπτύξεις όπως οι πάντα ενεργοί και οι conditional triggers. Όπου εξαρτώνται από τη σύνδεση τους με το αρχικό κύκλωμα για να ενεργοποιηθούν. Αυτοί οι ενεργοποιητές μπορούν να εισαχθούν κατά τα στάδια δρομολόγησης και να συνδεθούν αργότερα για την ενεργοποίησή τους.

Όπου και στις δύο περιπτώσεις ενεργοποιεί το payload που είναι η κακόβουλη ενέργεια που διαταράσσει την κανονική λειτουργία του κυκλώματος ή διαρρέει πληροφορίες στον εισβολέα.

2.2 Αναλυτικά η Επίθεση

Όπως έχουμε ήδη αναφερθεί για το μοντέλο της επίθεσης, σε αυτή την ενότητα θα περιγράψουμε τη ροή (flow) της επίθεσης. Η επίθεση αποτελείται από 2 στάδια:

- Εισαγωγή και Δρομολόγηση του Trojan
- Ενεργοποίηση του Trojan

Για την εισαγωγή του Trojan είχαμε αναφερθεί και στο προηγούμενο στάδιο που περιγράψαμε τα αρχεία που χρειάστηκαν για τη διαδικασία εισαγωγής και ουσιαστικά για τα πρώτα βήματα που θα έπρεπε να καταχωρηθούν για τη συνέχεια της επίθεσης. Αφού γίνει η εισαγωγή του Trojan, το Trojan συνδέεται σε συγκεκριμένο σημείο του FPGA που λέγεται Programmable Interconnect Point (PIP). Παρόλο, που γίνεται η σύνδεση στη συνέχεια έχουμε μία άμεση αποσύνδεση από το εργαλείο PnR. Το PnR (Place-and-Route) είναι μια διαδικασία στη σχεδίαση FPGA και ASIC, όπου οι λογικές μονάδες τοποθετούνται σε συγκεκριμένα σημεία της φυσικής διάταξης του FPGA και στη συνέχεια συνδέονται μεταξύ τους με διαδρομές. Στόχος είναι να εξασφαλιστεί ότι το κύκλωμα λειτουργεί σωστά, ακολουθώντας τις περιορισμένες δυνατότητες του FPGA. Παρόλο που στη δική μας περίπτωση σχεδιάζουμε το PnR για μία επίθεση.

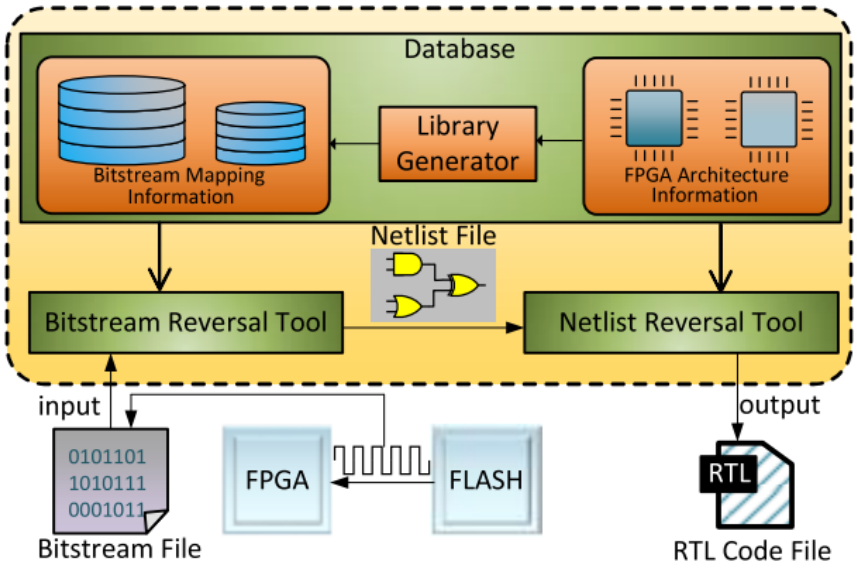
Το σημείο που γίνεται η σύνδεση του Trojan ονομάζεται TPIP. Όπου το TPIP δεν λειτουργεί σε περίπτωση που δεν είναι συνδεδεμένο. Όπου περιέχει και έναν trigger που ενεργοποιείται ανάλογα με τη διαδικασία επίθεσης όπου θα το περιγράψουμε παρακάτω.

Όπως αναφερθήκαμε προηγουμένως, τώρα ενεργοποιείται το Trojan στο FPGA που το εργαλείο προγραμματισμού είναι υπεύθυνο και για το bitstream. Όταν το εργαλείο προγραμματισμού συναντήσει το αποσυνδεδεμένο TPIP στο bitstream, θα το επανασυνδέσει. Μόλις το TPIP συνδεθεί, ο Trojan ενεργοποιείται. Στη συνέχεια, με την επανασύνδεση το TPIP ουσιαστικά επιτρέπει να γίνει η σύνδεση πλήρως (payload, trigger, circuit).

Στην επίθεση αυτή, η αρχική διαδικασία σχεδίασης του FPGA δεν αλλάζει. Στο επόμενο βήμα, όταν το σχέδιο τοποθετείται και δρομολογείται, προστίθεται το Trojan με έναν τρόπο που μπορεί να απενεργοποιηθεί προσωρινά. Έτσι, το Trojan παραμένει «κρυφό» και το FPGA λειτουργεί κανονικά, χωρίς να ανιχνεύεται, μέχρι να ενεργοποιηθεί σε ένα επόμενο στάδιο, όταν το σημείο αποσύνδεσης (TPIP) επανασυνδεθεί.

Το πλεονέκτημα της εισαγωγής Trojan μετά τη σύνθεση είναι ότι μπορεί να μολύνει το σχέδιο ακόμα και αν το IP core που παρέχεται από τρίτο μέρος είναι ήδη σε επίπεδο πυλών . Οι μέθοδοι που χρησιμοποιούν μηχανική μάθηση για να αναλύσουν το bitstream και να βρουν Trojan, συνήθως εντοπίζουν μόνο τα δίκτυα ενεργοποίησης (triggers).

Αλλά το Trojan που εισάγουμε δεν θα προκαλέσει αλλαγές στη συμπεριφορά του κυκλώματος μέχρι να ενεργοποιηθεί, έτσι δεν μπορεί να ανιχνευτεί σε αυτό το στάδιο. Επίσης, οι μέθοδοι που ψάχνουν για αχρησιμοποίητα δίκτυα μπορεί να εντοπίσουν κάτι ύποπτο, αλλά δεν μπορούν να εντοπίσουν την κακόβουλη αλλαγή στο κύκλωμα γύρω από τον Trojan. Ο Trojan παραμένει συνδεδεμένος με το κύκλωμα μέχρι το επόμενο στάδιο, και δεν μπορεί να εντοπιστεί κατά τη διάρκεια της τοποθέτησης και δρομολόγησης.



Εικόνα 3. Διαδικασία Reverse Engineering από Bitstream σε RTL

Συνολικά, τα εργαλεία αντίστροφης μηχανικής θα μπορούσαν να χρησιμοποιηθούν για να αναλύσουν και να ανιχνεύσουν Trojans, αλλά μπορεί να παραλείψουν κάποιες κακόβουλες αλλαγές, όπως αυτές που εισάγονται από τον Trojan στο FPGA, επειδή επικεντρώνονται στην αντίστροφη μηχανική του bitstream και του netlist, και όχι στην ανίχνευση αδρανών Trojans στη ροή σχεδίαση. Όπως μπορούμε να παρατηρήσουμε και στην εικόνα 3.

Για τη σωστή στιγμή της ενεργοποίησης του Trojan χρησιμοποιούμε έναν αλγόριθμο όπου η διαδικασία περιλαμβάνει δύο βασικά βήματα. Στο πρώτο, το εργαλείο δημιουργίας bitstream δημιουργεί το αρχείο διαμόρφωσης με τον Trojan, αλλά ο Trojan παραμένει αδρανής και αποσυνδεδεμένος. Στο δεύτερο βήμα, όταν το FPGA προγραμματίζεται, ο Trojan ενεργοποιείται όταν το εργαλείο προγραμματισμού συνδέει τον Trojan ξανά, γράφοντας ένα «1» στο bit διαμόρφωσης TPIP. Αυτή η διαδικασία περιγράφεται στον Αλγόριθμο 1, όπου ο Trojan εισάγεται και παραμένει αδρανής μέχρι το τελικό στάδιο του προγραμματισμού, όταν και ενεργοποιείται.

Στην επίθεση αυτή, στοχεύουμε σε ένα netlist ενός κυκλώματος AES και το τροποποιούμε για να διαρρεύσουμε το μυστικό κλειδί. Ο αλγόριθμος πρώτα ελέγχει αν το netlist περιέχει το κυκλωμα AES και αν είναι έγκυρο στόχος.

Στη συνέχεια, εισάγει έναν MUX ανάμεσα στα registers εξόδου του κυκλώματος και τις εξόδους του, ώστε να μπορούμε να στείλουμε το διαρρεύσαν κλειδί αντί για το κρυπτογράφημα. Ο αλγόριθμος επαναλαμβάνει αυτή τη διαδικασία για 8 bits του κλειδιού, και όσο η συνθήκη είναι αληθής, τα δεδομένα του κλειδιού συνδέονται με τις εξόδους του κυκλώματος. Αυτό επιτρέπει τη διαρροή του κλειδιού χωρίς να γίνει αντιληπτό από τις κανονικές λειτουργίες του κυκλώματος. Δηλαδή κατά κάποιο τρόπο έχουμε μπει στο σύστημα του FPGA και δεν «μας έχουν πάρει χαμπάρι».

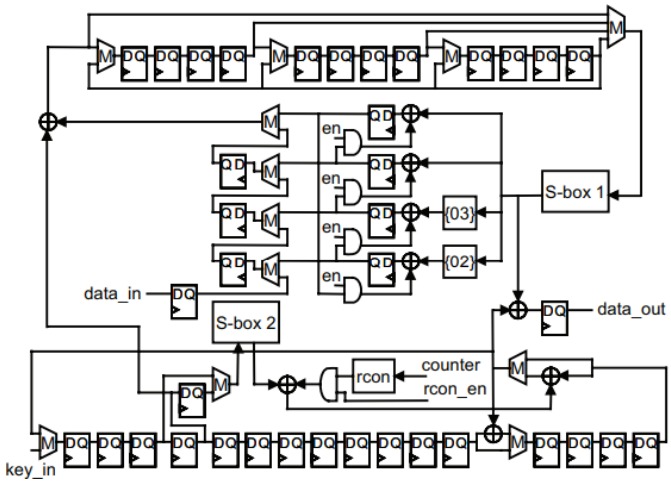
Ο Αλγόριθμος 2 εξηγεί πώς ενεργοποιείται το Trojan αφού το FPGA προγραμματιστεί. Όταν το εργαλείο προγραμματισμού φορτώνει το bitstream, αναζητά το TPIP και το επανασυνδέει. Μόλις γίνει αυτό, το Trojan ενεργοποιείται και μπορεί να διαρρεύσει το μυστικό κλειδί μέσω μιας θύρας I/O του FPGA. Έτσι, το Trojan παραμένει αδρανής μέχρι την τελευταία στιγμή και ενεργοποιείται μόνο όταν το FPGA προγραμματιστεί, επιτρέποντας στον επιτιθέμενο να κλέψει το κλειδί χωρίς να γίνει αντιληπτό.

Ο μηχανισμός που χρησιμοποιείται πίσω από την επικοινωνία είναι αναλόγως την περίπτωση [6]. Με απλά λόγια, η τοποθεσία του TPIP bit αναφέρεται στο συγκεκριμένο σημείο στο σχέδιο του FPGA όπου ο Trojan μπορεί να ενεργοποιηθεί. Οι μηχανισμοί επικοινωνίας που αναφέρονται είναι τρόποι για τον επιτιθέμενο να βρει ακριβώς πού βρίσκεται αυτή η τοποθεσία. Έχουμε 2 τρόποι επικοινωνίας, η ρητή επικοινωνία (explicit communication) που είναι ότι ο επιτιθέμενος αποκτά άμεσα αυτή την τοποθεσία από τα εργαλεία (tools) που χρησιμοποιούνται στη διαδικασία σχεδίασης. Η ύπουλη επικοινωνία (implicit communication) που σημαίνει ότι ο επιτιθέμενος κρύβει την τοποθεσία σε ανεπίσημα ή αδιάφορα μέρη του bitstream, έτσι ώστε τα εργαλεία να μην την εντοπίζουν εύκολα.

Μόλις ο επιτιθέμενος μάθει πού βρίσκεται το TPIP bit, μπορεί να το χρησιμοποιήσει για να ενεργοποιήσει τον Trojan κατά το προγραμματισμό του FPGA, αλλάζοντας το bit για να συνδέσει τον Trojan με το αρχικό κύκλωμα, κάνοντας το λειτουργικό.

2.3 Αποκάλυψη Κρυφών Πληροφοριών στο Σύστημα

Αρχικά, επιδεικνύουμε την επίθεση μας υλοποιώντας έναν AES core με διασύνδεση δεδομένων 8 bit. Όπου για την επίθεση θα χρησιμοποιήσουμε τη συσκευή iCE40HX-1k [7] όπου για την κρυπτογράφηση έχουμε το κύκλωμα στην εικόνα 4. Η εικόνα δείχνει πώς δουλεύει το κύκλωμα AES για την κρυπτογράφηση. Ένα μπλοκ δεδομένων και το κλειδί κρυπτογράφησης φορτώνονται ταυτόχρονα στο κύκλωμα, ένα byte τη φορά, μέσω των θυρών εισόδου. Στο πρώτο βήμα, το κύκλωμα προσθέτει το κλειδί στο μπλοκ δεδομένων με μια λειτουργία XOR. Στη συνέχεια, εκτελούνται 10 γύροι κρυπτογράφησης και στο τέλος το κρυπτογραφημένο μπλοκ δεδομένων εξάγεται από το κύκλωμα byte προς byte. Καθώς το κλειδί κρυπτογράφησης αλλάζει σε κάθε γύρο, πρέπει να το παρέχουμε ξανά στο κύκλωμα μαζί με νέο μπλοκ δεδομένων για την επόμενη κρυπτογράφηση [8].



Εικόνα 4. Κύκλωμα κρυπτογράφησης AES

Μετά τις διαδικασίες απόκρυψης και ενεργοποίησης του Trojan όπως έχουμε περιγράψει και παραπάνω όπου το μυστικό κλειδί διαρρέει μέσω μιας θύρας I/O. Συνεχίζει η επίθεση με την καταστροφή των EDA tools.

Η διαδικασία αυτή εκμεταλλεύεται αδυναμίες στα εργαλεία, παρακάμπτοντας τις μεθόδους επαλήθευσης μέσω του bitstream και της προ-διαμόρφωσης του FPGA, επιτρέποντας την αόρατη εισαγωγή και την ενεργοποίηση του Trojan. Ο Trojan παραμένει αδρανής και χωρίς καμία δραστηριότητα, μέχρι να προγραμματιστεί το FPGA και να επανασυνδεθεί το TPIP, ενεργοποιώντας το Trojan.

Αυτή η τεχνική, σε συνδυασμό με την προγραμματισμένη ενεργοποίηση μέσω του TPIP, καθιστά την επίθεση σχεδόν αόρατη κατά τη διάρκεια του προγραμματισμού του FPGA, προσφέροντας στον επιτιθέμενο πλήρη έλεγχο της διαρροής του μυστικού κλειδιού. Όταν ενεργοποιηθεί, το Trojan αποκαλύπτει το κλειδί του AES και το διαρρέει μέσω των θυρών I/O του FPGA, επιτρέποντας στον επιτιθέμενο να αποκτήσει πρόσβαση σε ευαίσθητες πληροφορίες χωρίς να ανιχνευθεί από τις παραδοσιακές μεθόδους ασφάλειας.

Όλες οι τεχνικές που έχουν χρησιμοποιηθεί μπορούν να υλοποιηθούν εύκολα και από άλλους φοιτητές αφού τα εργαλεία που έχουμε στην εργασία είναι open-source:

- Bitstream: IceStorm [34]
- Synthesis: Yosys [35]
- PnR (Place and Route): Arache-PnR [36] (όπου έχει αντικατασταθεί από nextPNR)

Στο πρώτο στάδιο της ροής σχεδίασης, χρησιμοποιείται το Yosys για τη σύνθεση του σχεδίου. Το Yosys παίρνει την περιγραφή του κυκλώματος και τη μετατρέπει σε netlist (αρχεία .blif), το οποίο είναι το επόμενο βήμα για την τοποθέτηση και δρομολόγηση. Στο στάδιο αυτό, το κύκλωμα παραμένει αμετάβλητο και το Trojan δεν είναι ενεργό.

Στο δεύτερο στάδιο, χρησιμοποιείται το Arachne-pnr για την τοποθέτηση και δρομολόγηση του κυκλώματος. Το Arachne-pnr διαβάζει το .blif netlist που παράχθηκε στο προηγούμενο στάδιο και τοποθετεί τα λογικά μπλοκ του FPGA, ενώ ταυτόχρονα καθορίζει τις συνδέσεις μεταξύ τους.

Σε αυτό το στάδιο εισάγεται το Trojan, Ο Trojan παραμένει ανενεργός και αποσυνδεδεμένος κατά τη διάρκεια της τοποθέτησης και δρομολόγησης. Χρησιμοποιείται το Arachne-pnr για να γίνει η εισαγωγή του Trojan σε ένα TPIP όπου θα είναι αόρατο.

Στο τελευταίο στάδιο, το IceStorm χρησιμοποιείται για τον προγραμματισμό του FPGA με το bitstream που έχει δημιουργηθεί. Στο στάδιο αυτό, ο Trojan ενεργοποιείται όταν το εργαλείο προγραμματισμού TPIP, το οποίο είναι αποσυνδεδεμένο στο bitstream, αλλά επανασυνδέεται κατά την ενεργοποίηση του FPGA.

3 Αξιολόγηση Αποτελεσμάτων και Ανάλυση

Αρχικά, για να ξεκινήσουμε την αξιολόγηση των αποτελεσμάτων θα πρέπει να αναφερθούμε ότι η ανίχνευση των Trojan είναι πάρα πολύ δύσκολη. Ο κυρίως λόγος αυτής της λογικής είναι το μέγεθος του κυκλώματος. Το κύκλωμα που έχουμε χρησιμοποιήσει είναι αρκετό μικρό αφού δεν μπορεί να γίνει η ανίχνευση από τις αναλύσεις που γίνονται με το side-channel.

Παρόλο που οι ανιχνεύσεις είναι δύσκολες λόγο του Trojan υπάρχει μία καθυστέρηση σε σύγκριση με κύκλωμα που δεν περιέχει κάποιο Trojan. Οι διαφορές αυτών των μετρήσεων είναι τόσο μικρές που είναι σχετικά πάρα πολύ δύσκολο η ανίχνευση και η κατανόηση ότι το κύκλωμα έχει κάποιο Trojan.

Resource name	Utilized by AES design			
	Available	Original	Malicious	Difference (δ)
# of LUTs (4-input)	1280	543	551	$1.45_{\%}$
# of FFs	1280	319	319	0
Delay (ns)	-	19.82	19.97	$0.76_{\%}$
Dynamic Power (mW)	-	10.246	10.448	$1.96_{\%}$

Εικόνα 5. Αποτελέσματα μετρήσεων αναλόγως της κατάστασης του κυκλώματος (με Trojan, χωρίς Trojan)

Για καλύτερη και αναλυτική κριτική μπορούμε να συγκρίνουμε τα αποτελέσματα που έχουμε στην εικόνα 5. Όπου η διαφορά μεταξύ των 2 περιπτώσεων είναι μόνο 8 LUTs που με το μέγεθος του κυκλώματος αυτό είναι στα περίπου 1.4%.

Σχετικά με την ενέργεια που καταναλώνεται, η διαφορά και εδώ είναι πάρα πολύ μικρή. Η διαφορά 1.96% στην κατανάλωση ενέργειας, δηλαδή η αύξηση στην κατανάλωση είναι πολύ μικρή. Αυτό δείχνει ότι το Trojan έχει ελάχιστη επίδραση στην κατανάλωση ενέργειας του FPGA, κάνοντάς το πολύ δύσκολο να ανιχνευτεί. Η μικρή αυτή αύξηση ενέργειας κάνει το Trojan πιο αόρατο σε σύγκριση με άλλα Trojan κυκλώματα που προκαλούν μεγαλύτερη αύξηση κατανάλωσης.

Αν οι barrier gates τοποθετηθούν σε μη κρίσιμη διαδρομή του κυκλώματος, δεν θα επηρεάσουν την καθυστέρηση. Αντίθετα, ακόμα και αν μια barrier gate, τοποθετηθεί στην κρίσιμη διαδρομή, η επίδραση στην καθυστέρηση θα είναι πολύ μικρή και δύσκολο να εντοπιστεί.

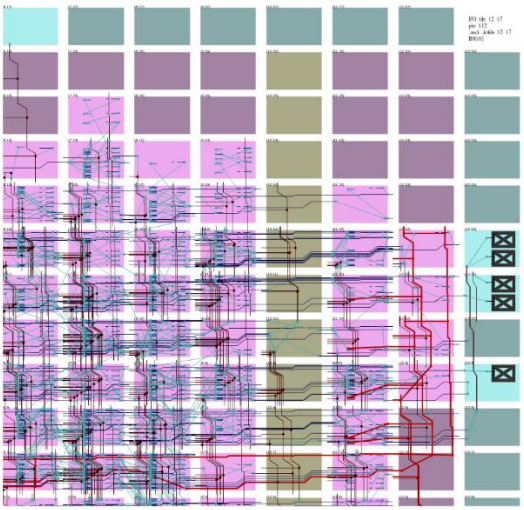
Στην περίπτωση μας, όταν τοποθετήσαμε την πύλη φραγής στην κρίσιμη διαδρομή, η καθυστέρηση του κυκλώματος αυξήθηκε μόνο κατά 0.15ns, δηλαδή η αύξηση είναι περίπου 0.76%. Αυτό σημαίνει ότι η επίδραση στην ταχύτητα του κυκλώματος είναι σχεδόν αμελητέα.

3.1 Πλεονεκτήματα

Τα πλεονεκτήματα που θα παρουσιάσουμε ουσιαστικά αφορούν την όψη του Trojan δηλαδή την επίθεση και την ευκολία της επίθεσης σε κυκλώματα αφού η εργασία μας γενικά βασίζεται στην επίθεση.

Το αρχικό πλεονέκτημα που περιέχουμε είναι ότι το Trojan μπορεί να τοποθετηθεί αυτόματα κατά τη διάρκεια του σχεδιασμού. Δηλαδή το Trojan δεν ενσωματώνεται από κάποια εξωτερική βοήθεια. Με αποτέλεσμα ο επιτιθέμενος χρειάζεται απλά να επηρεάσει τα εργαλεία σχεδίασης.

Άλλο πλεονέκτημα που έχουμε είναι, ότι ο επιτιθέμενος μπορεί να ξεκινήσει επιθέσεις σε περιοχές ταυτόχρονα με χρήση του Trojan. Που μπορεί να επηρεάσει πολλούς στόχους σε μεγάλη κλίμακα. Στην εικόνα 6 φαίνεται ένα παράδειγμα από την επίθεση μας που ουσιαστικά η επίθεση διασκορπίζεται παντού.



Εικόνα 6. Η διαδικασία επίθεσης με το Activation Route που διασκορπίζεται παντού

Ένα άλλο πλεονέκτημα που έχουμε αναλύσει και προηγουμένως είναι ότι η επίθεση παραμένει αόρατη σε περιπτώσεις που θα ενεργοποιηθεί αργότερα. Το Trojan μπορεί να μην είναι εμφανές και το παραγόμενο αποτέλεσμα να θεωρηθεί έγκυρο.

Το Trojan ενεργοποιείται μετά την ολοκλήρωση της διαμόρφωσης του FPGA, κάτι που τον καθιστά πιο ύπουλο και πιο δύσκολο να ανιχνευτεί από τις παραδοσιακές μεθόδους ανίχνευσης

Το Trojan χρησιμοποιεί το μηχανισμό ενεργοποίησης T(PIP) ακόμα και εξωτερικό ενεργοποιητή (I/O), που είναι κάτι που καθιστά τον Trojan πιο ευέλικτο και δυσκολότερο να εντοπιστεί. Η χρήση εξωτερικών ενεργοποιητών ή δυναμικών σημείων σύνδεσης βοηθά να παραμείνει κρυμμένος έως την ενεργοποίησή του.

Ο Trojan μας είναι σημαντικά μικρότερος σε μέγεθος σε σύγκριση με άλλες προηγούμενες υλοποιήσεις, γεγονός που τον καθιστά πιο δύσκολο να εντοπιστεί από τα σύγχρονα εργαλεία ανίχνευσης. Το μικρό μέγεθος μειώνει την επίδραση που έχει στο σχεδιασμό του κυκλώματος και καθιστά την ανίχνευσή του πιο δύσκολη.

3.2 Μειονεκτήματα

Στα μειονεκτήματα θα περιγράψουμε τους τρόπους ανίχνευσης τέτοιων επιθέσεων αλλά και τις αδύνατες πλευρές του κυκλώματος.

Το Trojan μπορεί να ανιχνευτεί και με τη χρήση του machine learning, υπάρχουν εργαλεία που μπορούν να μας δώσουν αποτελέσματα σχετικά με το bitstream του FPGA έτσι ώστε να καταλάβουμε αν το κύκλωμα μας είναι Trojan-free. Αυτή η μέθοδο έχει πάρα πολύ μεγάλη επιτυχία. Σύμφωνα με την έρευνα [9].

Ουσιαστικά, η μεθοδολογία που ακολουθείται για την ανίχνευση κυκλωμάτων Trojan βασίζεται στη χρήση μηχανικής μάθησης για την ανάλυση του bitstream που δημιουργείται από τα εργαλεία σχεδίασης FPGA.

Αρχικά, εκπαιδεύονται με μοντέλα ML με δεδομένα από μικροεπεξεργαστές χωρίς Trojan και με Trojan, χρησιμοποιώντας χαρακτηριστικά όπως την κατανάλωση ενέργειας, το αριθμός των λογικών πυλών και τις επιδόσεις. Στη συνέχεια, το εκπαιδευμένο μοντέλο χρησιμοποιείται για να ελέγξει αν το τελικό bitstream περιέχει Trojan. Η επιτυχία της ανίχνευσης είναι υψηλή, με τα μοντέλα να πετυχαίνουν ακρίβεια πάνω από 98%, τόσο όταν ο Trojan ενεργοποιείται όσο και όταν παραμένει ανενεργός. Δηλαδή, υπάρχουν μέθοδοι που μπορούν με μεγάλη επιτυχία και ευστοχία να ανιχνεύσουν τα Trojan.

Σαν επόμενο μειονέκτημα που έχουμε θα μπορούσε να είναι ότι ο χρήστης μπορεί να ενεργοποιήσει (με τάση) τη θύρα που δεν έχει εφαρμοστεί στο πραγματικό σχέδιο. Παρόλο αυτό είναι πάρα πολύ

δύσκολο να συμβεί κάτι τέτοιο. Η προσθήκη του Trojan προκαλεί μια τόσο μικρή αύξηση στην κατανάλωση ενέργειας (μόνο 8 LUTs) που είναι σχεδόν αόρατη στις τεχνικές ανίχνευσης που βασίζονται στην ανάλυση κατανάλωσης ενέργειας. Αυτό καθιστά την ανίχνευσή του πολύ δύσκολη, ακόμα και όταν οι ανιχνευτές προσπαθούν να εντοπίσουν αλλαγές στην κατανάλωση ενέργειας. Που για το κύκλωμα η ανίχνευση είναι απίθανο.

4 Συγκρίσεις με Προηγούμενες Εργασίες

Θα μπορούσαμε να κάνουμε τη σύγκριση με την εργασία [9] που ασχολείται με την ανίχνευση των Trojan. Η εργασία μας, γενικά ασχολείται με την επίθεση και είναι λογικό που δεν έχει ασχοληθεί παραπάνω για την ασφάλεια του κυκλώματος. Ενώ η εργασία [9] προσπαθεί να βρει κάποια λύση για τις εξελιγμένες επιθέσεις.

Για παράδειγμα η εργασία που χρησιμοποιεί το Machine Learning σαν μέθοδο ανίχνευσης έχει τα δέντρα απόφασης που σε κάθε βήμα η ανίχνευση εξελίσσεται. Οι κόμβοι στο δέντρο απόφασης αντιπροσωπεύουν τα χαρακτηριστικά, ενώ οι κλάδοι αντιπροσωπεύουν τις τιμές που μπορεί να αναλάβουν τα χαρακτηριστικά. Τα φύλλα αντιπροσωπεύουν το αποτέλεσμα της ταξινόμησης. Ένας αλγόριθμος που χρησιμοποιείται που το γνωρίζουμε και από κοντά είναι ο ID3.

Η ένωση των 2 paper για μία καλύτερη προσέγγιση σαν γενικό κύκλωμα θα ήταν και πιο εφικτό σε γενικές γραμμές για την παρουσία μιας τέτοιας εργασίας.

Σε σύγκριση με την εργασία [10] που ασχολείται με την ανίχνευση και ανάλυση των επιθέσεων Trojan σε FPGA έχουμε κάποιες προσεγγίσεις με τη δική μας εργασία. Αρχικά θα πρέπει να αναφερθούμε για τα χαρακτηριστικά που διαθέτουμε στη δική μας εργασία έτσι ώστε να μπορούμε να έχουμε μία ποιο εύκολη σύγκριση με άλλες εργασίες. Για το Abstraction Level έχουμε το bitstream που ουσιαστικά είναι το χαμηλότερο επίπεδο και αντιπροσωπεύει το αρχείο διαμόρφωσης για ρύθμιση του FPGA.

Έπειτα για το μηχανισμό ενεργοποίησης που είναι ο τρόπος ενεργοποίησης του Trojan η εργασία μας περιέχει TPIP/εξωτερικός ενεργοποιητής όπου ενεργοποιείται κατά τη διάρκεια της διαμόρφωσης του FPGA.

Για μέγεθος περιέχουμε 8 παραπάνω LUTs και για την επίδραση (payload) έχουμε τη διαρροή του κλειδιού, όπου το Trojan διαρρέει bits του μυστικού κλειδιού από τη μονάδα AES. Όπου σαν πεδίο εφαρμογής έχουμε το FPGA με αποτέλεσμα να έχουμε ευρύτερη προσβασιμότητα και καλύτερη απόδοση στο να κρύψουμε το Trojan.

Αν ξανά γυρίσουμε στην εργασία [10] όπου είναι σε μέγεθος 97 LUTs και 11 flip-flops. Περιέχει εσωτερικό μηχανισμό ενεργοποίησης που είναι κάτι παρόμοιο με το I/O της εργασίας μας όπου μπορεί να ενεργοποιηθεί εσωτερικά αλλά και εξωτερικά. Στη συνέχεια το επίπεδο που εισάγεται το Trojan είναι σε RTL που σημαίνει ότι η ανίχνευση του είναι ποιο εύκολη αφού θα πρέπει να περάσει ακόμα από πολλά άλλα επίπεδα όπως η στατική ανάλυση αλλά και το verification. Ενώ το επίπεδο bitstream είναι το τελευταίο επίπεδο. Για την επίδραση έχουμε τη διαφθορά του AES. Που στην δική μας είχαμε τη διαρροή κλειδιού..

Η διαρροή του κλειδιού είναι καλύτερη από τη διαφθορά του AES γιατί σου δίνει άμεση πρόσβαση στα κρυπτογραφημένα δεδομένα. Αν έχεις το κλειδί σημαίνει ότι μπορείς να αποκρυπτογραφήσεις εύκολα το περιεχόμενο χωρίς να χρειάζεται να πειράξεις τον αλγόριθμο. Αντίθετα, η διαφθορά του AES είναι πιο δύσκολη, απαιτεί τεχνικές γνώσεις και δεν εγγυάται επιτυχία. Δηλαδή, το κλειδί είναι σαν να έχεις το σωστό κωδικό για ένα χρηματοκιβώτιο, ενώ η διαφθορά είναι σαν να προσπαθείς να το σπάσεις.

Η άλλη σύγκριση που μπορούμε να κάνουμε είναι με την [11] όπου κάνει την ανάλυση ευπάθειας εισαγωγής του Trojan. Όπου το Trojan εισάγεται στο επίπεδο του Layout που δεν είναι και η καλύτερη προσέγγιση για την εισαγωγή κάποιου Trojan. Έχει μέγεθος 11 gates παραπάνω για την εισαγωγή του

Trojan που σε σύγκριση με την εργασία μας είναι μία καλύτερη μέθοδος. Όπου σαν μέθοδο έχει την αλλαγή των χαρακτηριστικών. Στο άρθρο αναφέρεται σε τροποποιήσεις του κυκλώματος που γίνονται έτσι ώστε να μην ανιχνευτεί το Trojan. Για παράδειγμα, τοποθετούνται τμήματα του Trojan σε μέρη του κυκλώματος που δεν επηρεάζουν την απόδοση του συστήματος,. Αυτές οι αλλαγές κάνουν τον Trojan να παραμένει κρυφός και να μην ανιχνεύεται από τις συνήθεις μεθόδους δοκιμών. Σε σύγκριση με τη δική μας εργασία σίγουρα η ανίχνευση είναι ποιο δύσκολη αλλά από την άλλη πλευρά, το key leakage είναι σίγουρα ποιο επικίνδυνο. Το κύκλωμα πρακτικά δουλεύει σε ASIC κύκλωμα που δεν είναι και η καλύτερη λύση αφού δεν έχει την ιδιότητα προγραμματισμού πολλές φορές.

Η έρευνα [12] διαφορετικά από τις προηγούμενες εργασίες έχει το μηχανισμό Always-on που σε σχέση χρήση ενέργειας δεν είναι και η ποιο εφικτή μέθοδο. Αλλά και 49 παραπάνω πύλες. Σαν payload έχει το Power Side Channel που είναι μία μέθοδο που χρησιμοποιεί παραλλαγές στην κατανάλωση ενέργειας μιας συσκευής για να εξάγει μυστικές πληροφορίες, όπως κρυπτογραφικά κλειδιά ή δεδομένα που επεξεργάζονται. Έτσι ώστε, το Trojan να μπορεί να χειριστεί την κατανάλωση ενέργειας του συστήματος. Σε σύγκριση με τη διαρροή κλειδιού προσφέρει μια χαμηλή προσέγγιση, αλλά απαιτούν πιο περίπλοκη ανάλυση για εκμετάλλευση.

Στην εργασία [13] σαν payload έχουμε τις λειτουργικές/ μη λειτουργικές επιδράσεις. Οι λειτουργικές επιδράσεις αφορούν περιπτώσεις όπου το Trojan αλλάζει την κανονική λειτουργία του κυκλώματος. Για παράδειγμα, μπορεί να προκληθεί άρνηση εξυπηρέτησης (Denial-of-Service - DoS), δηλαδή το κύκλωμα να σταματήσει να λειτουργεί σωστά ή να παράγει λανθασμένα αποτελέσματα. Ένα χαρακτηριστικό παράδειγμα από την εργασία είναι η εισαγωγή ενός συνδυαστικού Trojan (combinational) το οποίο ενεργοποιείται όταν ικανοποιηθούν σπάνιες συνθήκες (rare nodes) και οδηγεί στην εισαγωγή μιας πύλης XOR που αντιστρέφει το αποτέλεσμα της εξόδου. Επίσης, υπάρχουν ακολουθιακά Trojans (sequential), τα οποία ενεργοποιούνται όταν πραγματοποιηθεί μια συγκεκριμένη αλληλουχία καταστάσεων — για παράδειγμα, μέσω ενός μετρητή 3-bit που ενεργοποιεί το φορτίο (payload) μετά από αρκετές ενεργοποιήσεις.

Από την άλλη πλευρά, οι μη λειτουργικές επιδράσεις δεν επηρεάζουν άμεσα τη λειτουργικότητα του κυκλώματος, αλλά σχετίζονται με διαρροή πληροφοριών και άλλες φυσικές παραμέτρους του συστήματος (side-channels). Ένα παράδειγμα τέτοιου Trojan είναι το MOLES (Malicious Off-Chip Leakage Enabled by Side-Channels) Trojan, το οποίο είναι always-on και διαρρέει εσωτερικά σήματα του κυκλώματος μέσω μιας προκαθορισμένης διαδρομής. Οι χρήστες του εργαλείου μπορούν να ορίσουν Trojans αυτού του τύπου παρέχοντας ένα template κυκλώματος διαρροής (leakage circuit) και τα σήματα που επιθυμούν να διαρρεύσουν.

Το Key Leakage ανήκει κυρίως στις μη λειτουργικές επιδράσεις, γιατί δεν αλλάζει άμεσα τον τρόπο που δουλεύει το κύκλωμα. Αντίθετα, σκοπός του είναι να διαρρεύσει κρυφά σημαντικές πληροφορίες, όπως ένα κλειδί ασφαλείας, χωρίς να γίνει αντιληπτό. Παρ’ όλα αυτά, μπορεί να ενσωματωθεί με τρόπο που μοιάζει "κανονικός", δηλαδή να ενεργοποιείται μόνο σε συγκεκριμένες περιπτώσεις και να δίνει έξοδο που φαίνεται φυσιολογική, αλλά στην πραγματικότητα περιλαμβάνει κρυμμένα τα δεδομένα.

Άλλη μία εργασία σε σύγκριση με τη δική μας είναι η [14]. Που παρόλο χρησιμοποιεί την ίδια τεχνική για την εισαγωγή του Trojan (bitstream). Ο τρόπος που προσπαθεί να ενεργοποιήσει και να λειτουργήσει το Trojan είναι διαφορετικός. Αρχικά για το Trojan έχει ένα μικρό μέγεθος, των ταλαντωτών δακτυλίου τριών σταδίων. Ουσιαστικά είναι ένα απλό ψηφιακό κύκλωμα που αποτελείται από έναν περιττό αριθμό από πύλες NOT συνδεδεμένους σε βρόχο. Ένας ταλαντωτής με τρία στάδια έχει τρεις τέτοιες πύλες σε κυκλική διάταξη και μόλις ενεργοποιηθεί, αρχίζει να ταλαντώνεται με συχνότητα που εξαρτάται από την καθυστέρηση διάδοσης των πυλών.

Ο τρόπος που επηρεάζει το κύκλωμα ο ταλαντωτής δεν είναι σχετικά με τον τρόπου λειτουργίας του κυκλώματος, δηλαδή δεν είναι κάποιο συγκεκριμένο κύκλωμα που να επηρεάζει το κύκλωμα με χρήση κάποιου εργαλείου. Καταναλώνει ενέργεια και ανεβάζει τη θερμοκρασία του chip, προκαλώντας θερμική καταπόνηση και επιταχυνόμενη γήρανση του υλικού.

Με αποτέλεσμα όταν προστέθηκαν 150 τέτοιοι ταλαντωτές σε ένα FPGA, η θερμοκρασία του κυκλώματος ανέβηκε πάνω από 160°C. Παρόλο που το κύκλωμα συνέχισε να λειτουργεί κανονικά χωρίς να χαλάσει άμεσα, αυτή η αύξηση της θερμοκρασίας επιτάχυνε τη φθορά των υλικών του. Αυτό σημαίνει ότι μειώθηκε σημαντικά ο χρόνος ζωής του κυκλώματος, κάτι που υπολογίστηκε θεωρητικά μέσω ενός μαθηματικού τύπου που δείχνει πόσο πιο γρήγορα «γερνάει» το υλικό σε υψηλές θερμοκρασίες. Με έναν έμμεσο τρόπο έχει «χαλάσει» το κύκλωμα. Ο κύριος τρόπος δημιουργίας βλάβης στη [14] είναι η θερμοκρασία. Δηλαδή το payload αλλά και το Trojan είναι τα ROs. Σε σύγκριση με τη δική μας εργασία, η δική μας εργασία κάνει μία τέτοια βλάβη για να γίνει δυσκολότερο η ανίχνευση του Trojan από το κύκλωμα στη συνέχεια της επίθεσης. Ουσιαστικά είναι και το πρώτο στάδιο επίθεσης.

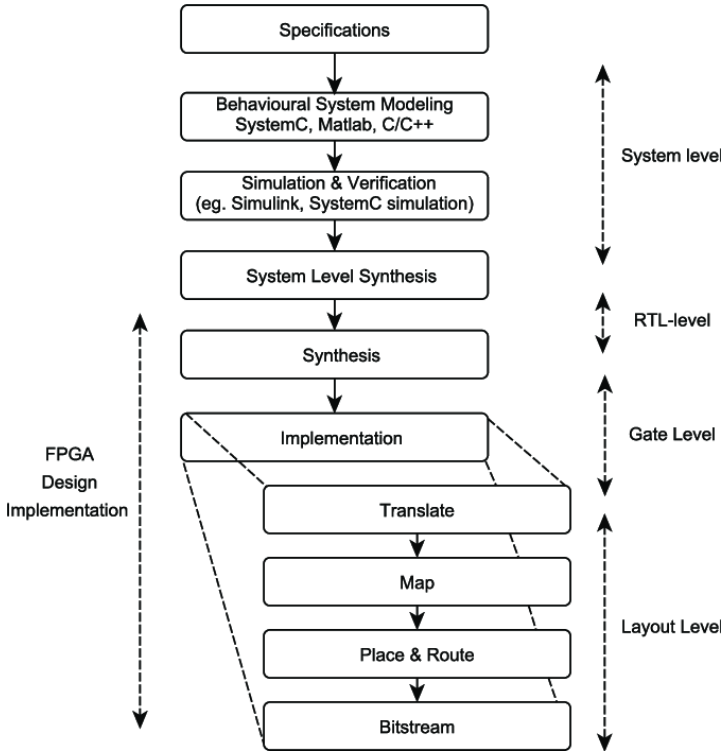
Διαφορετικά από την εργασία μας το Trojan ενσωματώνεται σε δύο επίπεδα στην εργασία [2]. Χρησιμοποιεί τα επίπεδα RTL και bitstream. Που σε σύγκριση με τη δική μας εργασία, ενσωματώνεται στο HDL και εξαπλώνεται μέσω παραβιασμένου εργαλείου σύνθεσης. Αυτός ο τρόπος κάνει πολύ δύσκολο να εντοπιστεί σε RTL/simulation τον Trojan, αλλά μπορεί να φανεί με bitstream checking. Που η δική μας εργασία δεν έχει κάποια ίχνη μετά την επίθεση. Βέβαια ανάλογα με την επίθεση που θέλουμε και οι δύο τρόποι μπορούν να γίνουν χρήσιμοι. Για μέγιστη μυστικότητα η δική μας εργασία ενώ για ευρύτερη διάδοση [2] θα μπορούσε να ήταν καλύτερη.

Το payload στη [2] ο αποκωδικοποιητής εντολών ενός επεξεργαστή ελέγχει αν μια εντολή χρειάζεται δικαιώματα υπερχρήστη (superuser). Αν δεν έχεις τα σωστά δικαιώματα, δεν μπορείς να την εκτελέσεις. Το Trojan αλλάζει αυτόν τον έλεγχο. Κατά τη διάρκεια της σχεδίασης, προστίθεται ένας LUT που φαίνεται φυσιολογικό στις προσομοιώσεις. Όμως, όταν το FPGA προγραμματιστεί, αυτό το κύκλωμα ενεργοποιείται και ακυρώνει τον έλεγχο δικαιωμάτων.

Έτσι, κάποιος μπορεί να εκτελέσει εντολές που κανονικά θα απαγορεύονταν. Δηλαδή, να πάρει παράνομα δικαιώματα υπερχρήστη (privilege escalation). Όλα αυτά γίνονται χωρίς να φαίνεται κάτι ύποπτο στη φάση σχεδίασης.

Η μεγαλύτερη διαφορά με το key leakage θα ήταν ότι στο privilege escalation μας δίνεται τα δικαιώματα εκτέλεσης του κυκλώματος, ενώ στο key leakage κλέβονται τα δεδομένα με χρήση του κλειδιού.

Τέλος, για καλύτερη ανάλυση των Abstraction Levels έχουμε την εικόνα 7 που μας εξηγεί κατά κάποιο τρόπο όλα τα επίπεδα και ότι το bitstream είναι το τελευταίο επίπεδο. Με αποτέλεσμα να είναι η καλύτερη επιλογή για την τοποθέτηση του Trojan.



Εικόνα 7. Τυπικά επίπεδα σχεδίασης FPGA και επίπεδα αφαίρεσης (Abstraction Level)

5 Βιβλιογραφία

1. Bhunia S, Hsiao MS, Banga M, Narasimhan S (2014) Hardware Trojan attacks: Threat analysis and countermeasures. Proc IEEE 102(8):1229–1247
2. Krieg C, Wolf C, Jantsch A (2016) Malicious LUT: a stealthy FPGA trojan injected and triggered by the design flow. In: 2016 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), IEEE Press, pp 1–8. https://doi.org/10.1145/2966986.2967054
3. Mal-Sarkar S, Karam R, Narasimhan S, Ghosh A, Krishna A, Bhunia S (2016) Design and validation for FPGA trust under hardware Trojan attacks. IEEE Trans Multi-Scale Comput Syst 2(3):186–198. https://doi.org/10.1109/TMSCS.2016.2584052
4. Tehranipoor M, Koushanfar F (2010) A survey of hardware Trojan taxonomy and detection. IEEE Des Test Comput 27(1):10–25. https://doi.org/10.1109/MDT.2010.7
5. Xiao K, Forte D, Jin Y, Karri R, Bhunia S, Tehranipoor M (2016) Hardware Trojans: Lessons learned after one decade of research. ACM Trans Des Autom Electron Syst 22(1):6. https://doi.org/10.1145/2906147
6. Ahmed QA, Wiersema T, Platzner M (2021) Malicious routing: circumventing bitstream-level verification for FPGAs. In: 2021 Design, Automation & Test in Europe Conference & Exhibition (DATE), pp 1490–1495. https://doi.org/10.23919/DATE51398.2021.9474026
7. Lattice Semiconductor. Lattice Semiconductor. Accessed 21 Nov 2022 http://www.latticesemi.com/iCE40.
8. Hamalainen P, Alho T, Hannikainen M, Hamalainen T (2006) Design and implementation of low-area and low-power AES encryption hardware core. In: 9th

EUROMICRO Conference on Digital System Design (DSD'06), pp 577–583. https://doi.org/10.1109/DSD.2006.40
9. Palumbo A, Cassano L, Luzzi B, Hernández JA, Reviriego P, Bianchi G, Ottavi M (2022) Is your FPGA bitstream hardware Trojan-free? Machine learning can provide an answer. J Syst Architect 128:102543. https://doi.org/10.1016/j.sysarc.2022.102543
10. Giri N, Anandakumar NN (2020) Design and analysis of hardware trojan threats in reconfigurable hardware. In: 2020 International Conference on Emerging Trends in Information Technology and Engineering (ic-ETITE), pp 1–5. https://doi.org/10.1109/ic-ETITE47903.2020.227
11. Salmani H, Tehranipoor MM (2016) Vulnerability analysis of a circuit layout to hardware trojan insertion. IEEE Trans Inf Forensics Secur 11(6):1214–1225. https://doi.org/10.1109/TIFS.2016.2520910
12. Lin L, Burleson W, Paar C (2009a) MOLES: malicious of-chip leakage enabled by side-channels. In: Proceedings of the 2009 International Conference on Computer-Aided Design - ICCAD '09, ACM Press, San Jose, California, pp 117. https://doi.org/10.1145/1687399.1687425
13. Cruz J, Huang Y, Mishra P, Bhunia S (2018) An automated configurable trojan insertion framework for dynamic trust benchmarks. In: 2018 Design, Automation & Test in Europe Conference & Exhibition (DATE), pp 1598–1603. https://doi.org/10.23919/DATE.2018.8342270
14. Chakraborty RS, Saha I, Palchaudhuri A, Naik GK (2013) Hardware Trojan insertion by direct modifcation of FPGA confguration bitstream. IEEE Des Test 30(2):45–54. https://doi.org/10.1109/MDT.2013.2247460
15. Cotton Seed. Arachne-pnr. https://github.com/YosysHQ/arachne-pnr. Accessed 15 Nov 2022
16. Nielsen K (2020) ICE40 layout viewer. [Online]. https://github.com/knielsen/ice40_viewer
17. Wolf C, Lasser M (2021) Project IceStorm. https://clifford.fm/icestorm
18. Wolf C. Yosys open synthesis suite. https://clifford.fm/ .Accessed 15 Dec 2022
19. Lorandel, J., Prévotet, J.-C., & Hélard, M. (2016). Fast power and performance evaluation of FPGA-based wireless communication systems. *IEEE Access*, 4, 1–1. https://doi.org/10.1109/ACCESS.2016.2559781